

Vienmatis optimizavimas

Lukaš Patrik Magalinski

Rugsėjis 2024

1) Tikslo Funkcija $f(x) = \frac{(x^2-a)^2}{b} - 1$

a ir b - studento pažymėjimo numerio paskutiniai du skaitmenys. Mano numeris: [redacted]. Iš to išplaukia, kad a = 0, b = 4.

Išstatome a ir b į tikslo funkciją: $f(x) = \frac{(x^2-0)^2}{4} - 1$

Suprastiname: $f(x) = \frac{(x^2)^2}{4} - 1 = \frac{x^4}{4} - 1$

Pasinaudojus intervalo dalijimo pusiau, aukšnio pjūvio, Niutono metodo vienmačio optimizavimo algoritmais, rasime lokulųjį minimumą tašką tikslo funkcijoje $f(x) = \frac{x^4}{4} - 1$ ir palyginsime algoritmų rezultatus. Minimumo tašką ieškosime $x \in [0, 10]$ intervale, dėl to l = 0, r = 10 ir L = 10. Kiekvienas algoritmas baigs savo darbą tik tada, kai intervalo ilgis bus mažesnis už $\epsilon = 0.0001$.

2) Intervalo dalijimo pusiau algoritmas

Šis metodas veikia taip: pirmiausia pasirenkamas intervalas [l,r], kur ieškome funkcijos minimumo. Intervalo vidurys x_m yra naudojamas kartu su dviem papildomais taškais x_1 ir x_2 , kurie yra tarp vidurio ir kraštinių intervalo ribų. Tada apskaičiuojamos funkcijos reikšmės $f(x_m)$, $f(x_1)$ ir $f(x_2)$, ir palyginamos, kad būtų galima atmesti vieną iš intervalo pusių, kur funkcija pasiekia didesnę reikšmę. Šis procesas kartojamas, kol intervalas sumažėja iki tam tikros mažos reikšmės ϵ , o tada gaunamas minimumo taškas.

Parašiau Python programą (žr. Priedas 6.1, interval_halving_method), kuri randa lokulųjį minimumo tašką naudodama intervalo dalijimo pusiau algoritmą. Algoritmui reikėjo 17 iteracijų ir 51 funkcijos $f(x)$ iškviatimų, kad būtų rastas lokalusis minimumo taškas. Dveiose lentelėse žemiau nurodžiau visų 17 iteracijų duomenis:

Bandymų taškai ir jų reikšmės						
Iteracija	x_1	$f(x_1)$	x_2	$f(x_2)$	x_m	$f(x_m)$
1	2.500000	8.765625	7.500000	790.015625	5.000000	155.250000
2	1.250000	-0.389648	3.750000	48.438477	2.500000	8.765625
3	0.625000	-0.961853	1.875000	2.089905	1.250000	-0.389648
4	0.312500	-0.997616	0.937500	-0.806881	0.625000	-0.961853
5	0.156250	-0.999851	0.468750	-0.987930	0.312500	-0.997616
6	0.078125	-0.999991	0.234375	-0.999246	0.156250	-0.999851
7	0.039062	-0.999999	0.117188	-0.999953	0.078125	-0.999991
8	0.019531	-1.000000	0.058594	-0.999997	0.039062	-0.999999
9	0.009766	-1.000000	0.029297	-1.000000	0.019531	-1.000000
10	0.004883	-1.000000	0.014648	-1.000000	0.009766	-1.000000
11	0.002441	-1.000000	0.007324	-1.000000	0.004883	-1.000000
12	0.001221	-1.000000	0.003662	-1.000000	0.002441	-1.000000
13	0.000610	-1.000000	0.001831	-1.000000	0.001221	-1.000000
14	0.000305	-1.000000	0.000916	-1.000000	0.000610	-1.000000
15	0.000153	-1.000000	0.000458	-1.000000	0.000305	-1.000000
16	0.000076	-1.000000	0.000229	-1.000000	0.000153	-1.000000
17	0.000038	-1.000000	0.000114	-1.000000	0.000076	-1.000000

Čia x_1 , x_2 , x_m yra bandymo taškai, o $f(x_1)$, $f(x_2)$, $f(x_m)$ jų reikšmės.

Intervalo pakeitimai				
Iteracija	l	r	L	Atmetamas intervalas
0	0.000000	10	10	Pradinis intervalas
1	0.000000	5.000000	5.000000	R
2	0.000000	2.500000	2.500000	R
3	0.000000	1.250000	1.250000	R
4	0.000000	0.625000	0.625000	R
5	0.000000	0.312500	0.312500	R
6	0.000000	0.156250	0.156250	R
7	0.000000	0.078125	0.078125	R
8	0.000000	0.039062	0.039062	R
9	0.000000	0.019531	0.019531	R
10	0.000000	0.009766	0.009766	R
11	0.000000	0.004883	0.004883	R
12	0.000000	0.002441	0.002441	R
13	0.000000	0.001221	0.001221	R
14	0.000000	0.000610	0.000610	R
15	0.000000	0.000305	0.000305	R
16	0.000000	0.000153	0.000153	R
17	0.000038	0.000114	0.000076	LR

l - kairiojo intervalo taškas. *r* - dešiniojo intervalo taškas. *L* - intervalo ilgis. Atmetamas intervalas - kurią intervalo dalį algoritmas atmetė (*L* - kairiąją, *R* - dešiniąją, *LR* - kairiąją ir dešiniąją).

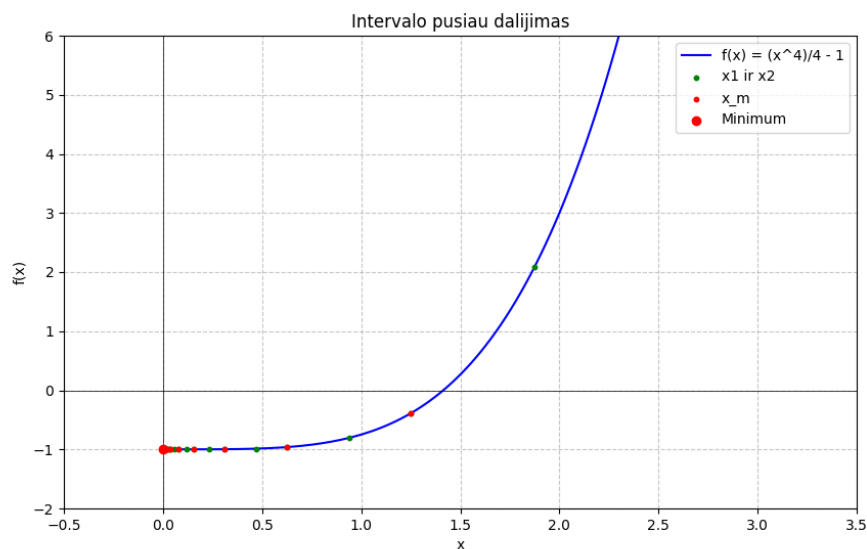
Po 17-os (18 su pradiniu intervalu) iteracijos (kai buvo atmetas LR intervalas), kairysis ir dešinysis taškai bei intervalo ilgis turės tokias reikšmes: $l = 0.000038$, $r = 0.000114$ ir $L = 0.000076$. Kadangi $L < \varepsilon$ ($\varepsilon = 1e-4$), daugiau iteracijų nereikėjo ir dėl to nėra 18 iteracijos.

Intervalo dalijimo pusiau algoritmas gražino tokį $x_{\min}$ tašką:

$$x_{\min} = 0.000076$$

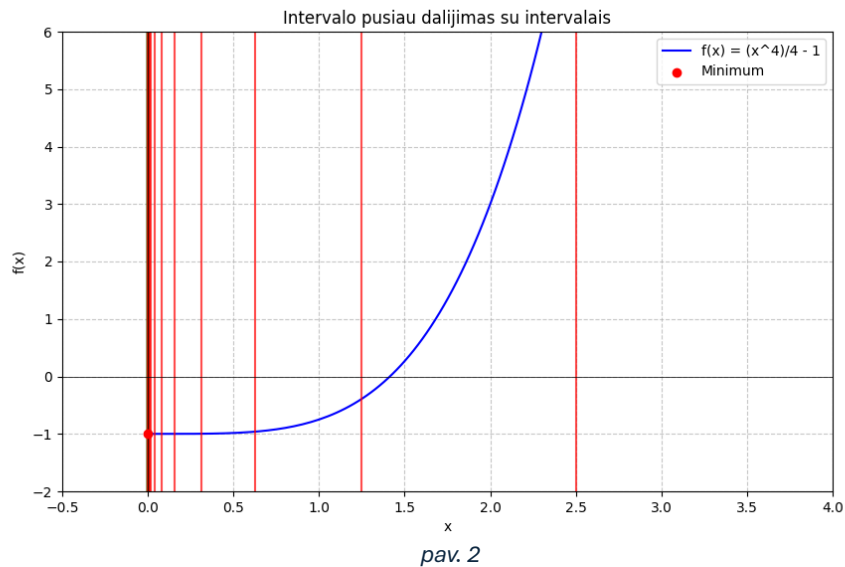
$$\min_{x \in [0;10]} f(x_{\min}) = \min_{x \in [0;10]} f(0.000076) \approx -1$$

Funkcijos grafikas su pavaizduotais x_1 , x_2 , x_m bandymo taškais:



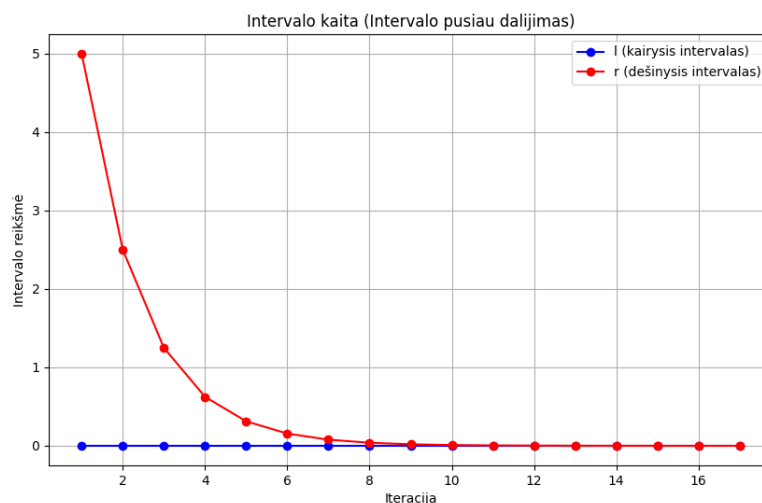
pav. 1

Intervalo pusiau dalijimo metodas su intervalais, mažėjančiais per iteracijas, ir funkcijos $\frac{x^4}{4} - 1$ kreivę:



Šis grafikas (pav. 2) rodo intervalo pusiau dalijimo metodą su intervalais, mažėjančiais per iteracijas. Iteracijos metu algoritmas atmeta dešinę intervalo pusę ir susiaurina intervalą. Kiekviena nauja vertikali linija parodo, kaip mažėja dešinė intervalo riba r . Kairysis taškas l yra beveik pastovus ir labai arti 0, todėl dauguma linijų yra sutelkta šalia $l=0$. Dėl šios priežasties linijos yra labai arti viena kitos ties kairiąja puse, nes l beveik nesikeičia, o r nuolat artėja prie 0.

Grafikai (pav. 1 ir pav. 2) parodo, kaip intervalo pusiau dalijimo metodas efektyviai mažina paieškos intervalą ir artėja prie sprendimo. Intervalo pusiau dalijimas su intervalais (pav. 2) grafikas vizualizuoja intervalų susiaurėjimą, o grafikas Intervalo pusiau dalijimas (pav. 1) rodo, kaip funkcijos taškai per kiekvieną iteraciją artėja prie minimumo.



Raudona linija atvaizduoja dešinįjį intervalą (r). Matome, kad per kiekvieną iteraciją dešinysis taškas r mažėja. Tai reiškia, kad algoritmo metu visos iteracijos atmetė dešinę pusę intervalo, o dešinysis taškas nuosekliai artėjo link minimumo. Taip aiškinama, kodėl r sparčiai mažėja. Mėlyna linija rodo kairįjį intervalą (l). Ši linija išlieka beveik pastovi ties nuliu, nes l (kairysis taškas) iš esmės nesikeičia, kai dešinė pusė visada yra atmetama. Kadangi optimizavimo užduotis prasideda nuo intervalo $[0, 10]$ ir minimumas yra ties 0, kairysis taškas l išlieka artimas minimumui per visą algoritmą.

3) Auskinio pjūvio algoritmas

Šis metodas taip pat skirtas intervalui mažinti, tačiau jis remiasi auksinio pjūvio santykiu $\tau \approx 0.618$. Iš pradžių pasirinkus intervalą $[l, r]$, apskaičiuojami du taškai x_1 ir x_2 pagal τ . Tada apskaičiuojamos funkcijos reikšmės šiuose taškuose, ir atmetama ta intervalo dalis, kurioje funkcijos reikšmė yra didesnė. Taip iteruojant, intervalas palaipsniui mažinamas tol, kol gaunamas pakankamai tikslus minimumo taškas. Skirtingai nuo intervalo dalijimo pusiau metodo, auksinio pjūvio metodas yra greitesnis ir efektyvesnis, nes naudoja optimalų pjūvį.

Parašiau Python programą (žr. Priedas 6.1, `golden_section_search`), kuri randa lokalųjį minimumo tašką naudodama auksinio pjūvio algoritmą. Algoritmui reikėjo 25 iteracijų ir 51 funkcijos $f(x)$ iškvietimų, kad būtų rastas lokalusis minimumo taškas. Dvejuose lentelėse žemiau nurodžiau visų 25 iteracijų duomenis:

Bandymų taškai ir jų reikšmės						
Iteracija	x_1	$f(x_1)$	x_2	$f(x_2)$	x_m	$f(x_m)$
1	3.819660	52.215591	6.180340	363.745084	5.000000	155.250000
2	2.360680	6.764050	3.819660	52.215591	3.090170	21.796568
3	1.458980	0.132760	2.360680	6.764050	1.909830	2.325974
4	0.901699	-0.834733	1.458980	0.132760	1.180340	-0.514747
5	0.557281	-0.975888	0.901699	-0.834733	0.729490	-0.929203
6	0.344419	-0.996482	0.557281	-0.975888	0.450850	-0.989671
7	0.212862	-0.999487	0.344419	-0.996482	0.278640	-0.998493
8	0.131556	-0.999925	0.212862	-0.999487	0.172209	-0.999780
9	0.081306	-0.999989	0.131556	-0.999925	0.106431	-0.999968
10	0.050250	-0.999998	0.081306	-0.999989	0.065778	-0.999995
11	0.031056	-1.000000	0.050250	-0.999998	0.040653	-0.999999
12	0.019194	-1.000000	0.031056	-1.000000	0.025125	-1.000000
13	0.011862	-1.000000	0.019194	-1.000000	0.015528	-1.000000
14	0.007331	-1.000000	0.011862	-1.000000	0.009597	-1.000000
15	0.004531	-1.000000	0.007331	-1.000000	0.005931	-1.000000
16	0.002800	-1.000000	0.004531	-1.000000	0.003666	-1.000000
17	0.001731	-1.000000	0.002800	-1.000000	0.002266	-1.000000
18	0.001070	-1.000000	0.001731	-1.000000	0.001400	-1.000000
19	0.000661	-1.000000	0.001070	-1.000000	0.000865	-1.000000
20	0.000409	-1.000000	0.000661	-1.000000	0.000535	-1.000000
21	0.000253	-1.000000	0.000409	-1.000000	0.000331	-1.000000
22	0.000156	-1.000000	0.000253	-1.000000	0.000204	-1.000000
23	0.000096	-1.000000	0.000156	-1.000000	0.000126	-1.000000
24	0.000060	-1.000000	0.000096	-1.000000	0.000078	-1.000000
25	0.000037	-1.000000	0.000060	-1.000000	0.000048	-1.000000

Čia x_1 ir x_2 yra bandymo taškai, o $f(x_1)$ ir $f(x_2)$ jų reikšmės. x_m - iteracijos surastas lokalusis minimumo taškas.

Intervalo pakeitimai				
Iteracija	l	r	L	Atmetamas intervalas
0	0.000000	10	10	Pradinis intervalas
1	0.000000	6.180340	6.180340	R
2	0.000000	3.819660	3.819660	R
3	0.000000	2.360680	2.360680	R
4	0.000000	1.458980	1.458980	R
5	0.000000	0.901699	0.901699	R
6	0.000000	0.557281	0.557281	R
7	0.000000	0.344419	0.344419	R
8	0.000000	0.212862	0.212862	R
9	0.000000	0.131556	0.131556	R
10	0.000000	0.081306	0.081306	R
11	0.000000	0.050250	0.050250	R
12	0.000000	0.031056	0.031056	R
13	0.000000	0.019194	0.019194	R
14	0.000000	0.011862	0.011862	R
15	0.000000	0.007331	0.007331	R
16	0.000000	0.004531	0.004531	R
17	0.000000	0.002800	0.002800	R
18	0.000000	0.001731	0.001731	R
19	0.000000	0.001070	0.001070	R
20	0.000000	0.000661	0.000661	R
21	0.000000	0.000409	0.000409	R
22	0.000000	0.000253	0.000253	R
23	0.000000	0.000156	0.000156	R
24	0.000000	0.000096	0.000096	R

l - kairiojo intervalo taškas. *r* - dešiniojo intervalo taškas. *L* - intervalo ilgis. Atmetamas intervalas - kurią intervalo dalį algoritmas atmetė (*L* - kairiąją, *R* - dešiniąją).

Po 24-os (25 su pradiniu intervalu) iteracijos (kai buvo atmestas LR intervalas), kairysis ir dešinysis taškai bei intervalo ilgis turės tokias reikšmes: $l=0.000000$, $r=0.000096$ ir $L=0.000096$. Kadangi $L < \varepsilon$ ($\varepsilon = 1e-4$), daugiau iteracijų nereikėjo ir dėl to nėra 25 iteracijos.

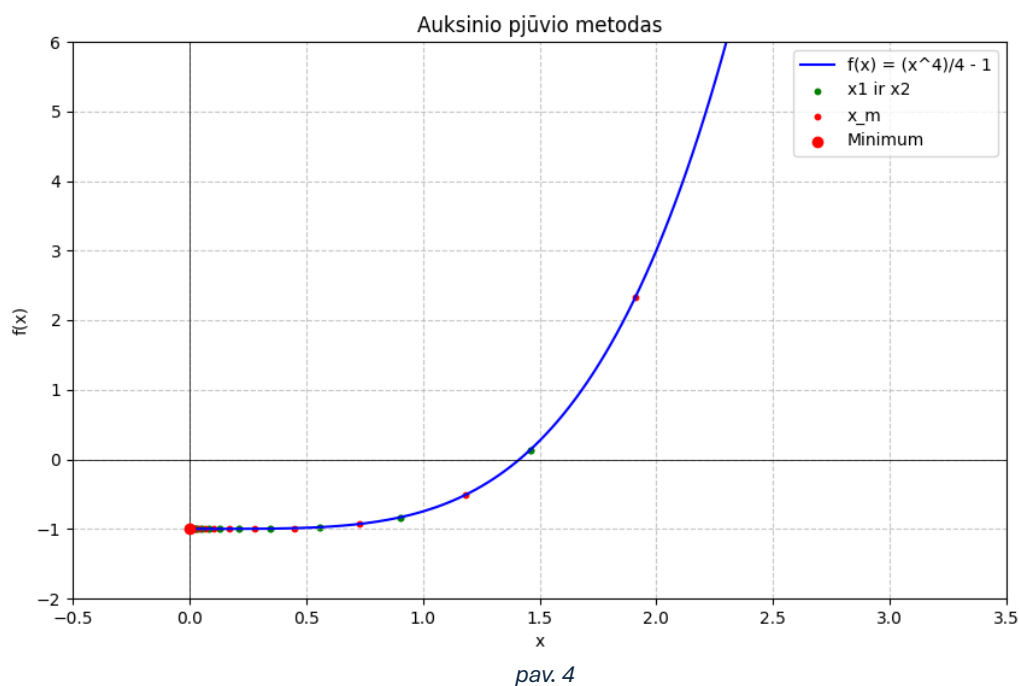
Auksinio pjūvio algoritmas gražino tokį $x_{\min}$ tašką:

$$x_{\min} = \frac{(l + r)}{2} = \frac{(0.000000 + 0.000096)}{2} = 0.000048$$

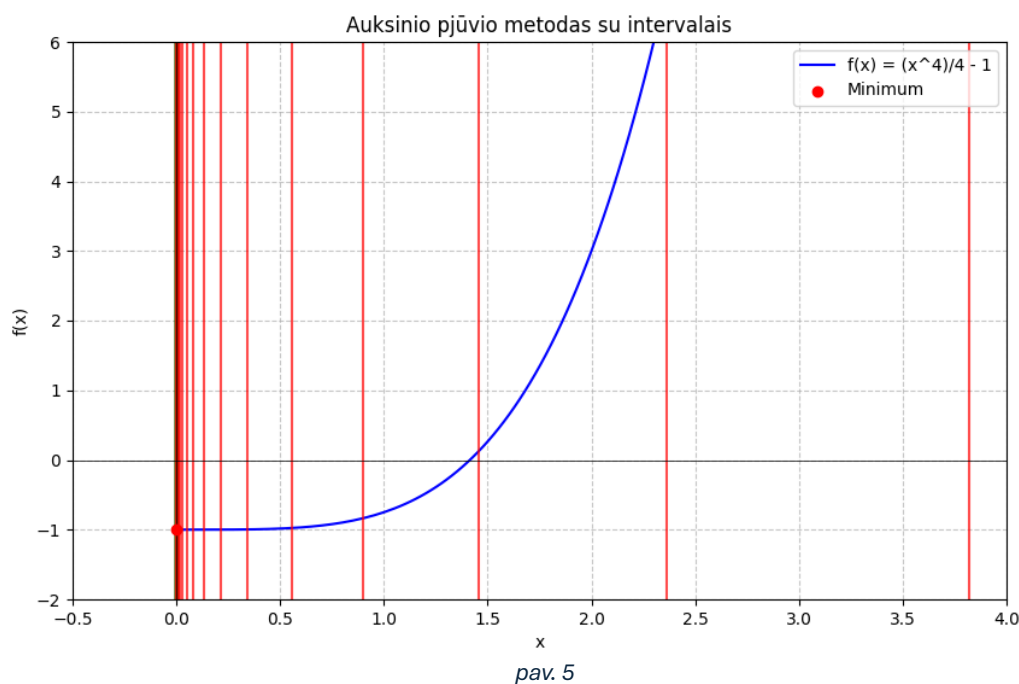
$$\min_{x \in [0;10]} f(x_{\min}) = \min_{x \in [0;10]} f(0.000048) \approx -1$$

Auksinio pjūvio metodas efektyviai sukonvergavo į tašką $x_{\min} = 0,000048$, kuris yra labai artimas $x=0$. Tai rodo, kad metodas sugebėjo tiksliai nustatyti intervalo sritį, kurioje yra funkcijos minimumas. Įstatę $x_{\min}$ į funkciją $f(x)$, gavome $f(x_{\min}) \approx -1$. Tai patvirtina, kad funkcijos minimalioji reikšmė intervale $[0,10]$ yra -1 , ir ji pasiekiamą ties $x=0$. Reikšmė $f(x_{\min}) \approx -1$ atitinka teorinius lūkesčius ir patvirtina, kad funkcija turi globalų minimumą ties $x=0$.

Funkcijos grafikas su pavaizduotais x_1 , x_2 , x_m bandymo taškais:

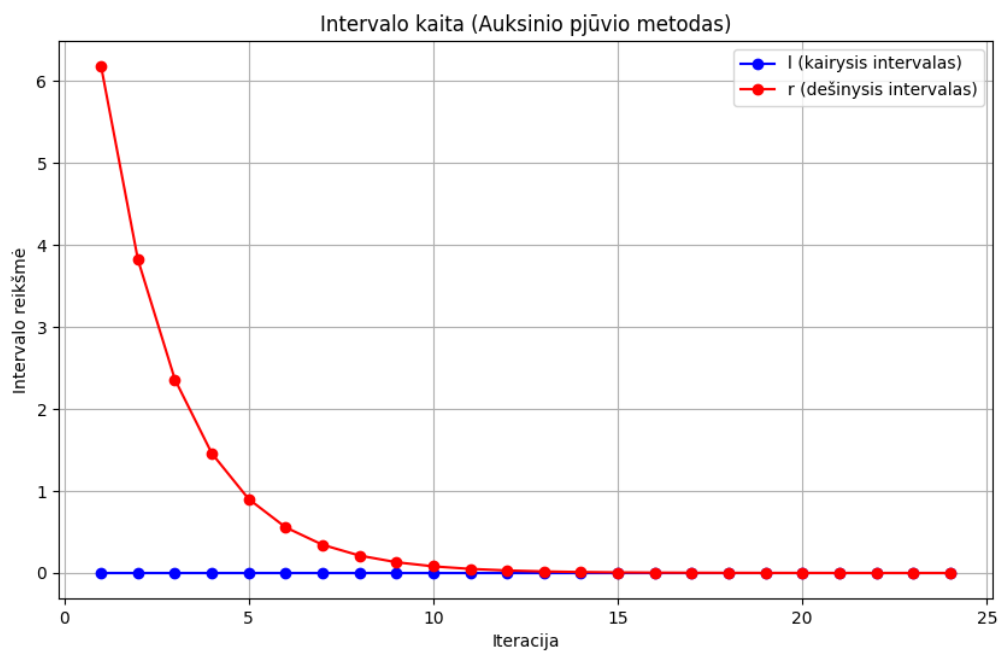


Intervalo pusiau dalijimo metodas su intervalais, mažėjančiais per iteracijas, ir funkcijos $\frac{x^4}{4} - 1$ kreivę:



Šis grafikas vaizduoja, kaip auksinio pjūvio metodas taikomas ieškant funkcijos minimumo. Mėlyna kreivė rodo funkciją $f(x) = \frac{x^4}{4} - 1$, o raudonas taškas pažymi minimumo tašką ties $x = 0$, kur $f(0) = -1$. Pradinis paieškos intervalas prasideda nuo $l = 0$ iki $r = 10$, o kiekvienoje iteracijoje intervalas yra mažinamas, eliminuojant tas sritis, kuriose nėra minimumo. Raudonos vertikalios linijos grafike parodo, kaip šios intervalo ribos palaipsniui susiaurėja, vis labiau sutelkiant paiešką aplink minimumo tašką. Algoritmas efektyviai ir greitai mažina paieškos erdvę, kol galutinai randa tašką, kuriame funkcija pasiekia savo minimumą.

Grafikuose (pav. 4, pav. 5) vaizduojamas auksinio pjūvio metodo taikymas funkcijos minimizavimui. Auksinio pjūvio metodas su intervalais grafike (pav. 5) matome, kaip per kiekvieną iteraciją palaipsniui mažėja paieškos intervalai, atvaizduojami raudonomis vertikaliomis linijomis. Šis grafikas rodo, kaip intervalai tampa siauresni, kol galų gale algoritmas susikoncentruoja ties minimumo vieta $x = 0$, kur $f(x) = -1$. Auksinio pjūvio metodas (pav. 4) grafike pavaizduoti kiekvienos iteracijos apskaičiuoti taškai x_1 , x_2 ir x_m ant funkcijos $f(x)$ kreivės. Algoritmas palaipsniui ieško minimumo, eliminuodamas mažiau svarbias sritis ir palikdamas tas, kuriose funkcijos reikšmė yra mažiausia. Raudonas taškas žymi tikrąjį funkcijos minimumą, kuris yra $x = 0$ ir $f(x) = -1$. Šis grafikas parodo, kaip kiekvienoje iteracijoje tikrinamos funkcijos vertės ir kaip greitai metodas konverguoja į minimalią vertę.



pav. 6

Šis grafikas vaizduoja auksinio pjūvio metodo intervalo kintamumo dinamiką, iliustruojant, kaip keičiasi kairysis (l) ir dešinysis (r) intervalai per kiekvieną iteraciją. Raudona linija atvaizduoja dešinįjį intervalą r , kuris kiekvienoje iteracijoje mažėja, kadangi algoritmo metu nuolat atmetama dešinioji intervalo pusė. Tai paaiškina, kodėl r reikšmė sparčiai mažėja ir artėja prie minimumo. Tuo tarpu mėlyna linija atspindi kairiojo intervalo l reikšmes, kurios beveik nesikeičia ir išlieka pastovios ties nulinio reikšmės. Kadangi pradinė intervalo reikšmė yra $[0, 10]$, o minimumas yra ties 0 , kairysis taškas l iš esmės nekinta, nes dešinioji pusė visada atmetama. Grafike matyti, kad kairysis intervalas (l) greitai konverguoja ir stabilizuojasi jau ankstyvoje metodo eigoje, o dešinysis intervalas (r) pradžioje yra didelis, bet sparčiai mažėja, kol abu intervalai tampa labai arti vienas kito, suformuojant siaurą intervalą aplink minimumo tašką. Tai iliustruoja, kaip efektyviai auksinio pjūvio metodas mažina paieškos erdvę, kol intervalai tiksliai apibrėžia funkcijos minimumą. Išvada: grafikas demonstruoja, kaip algoritmas greitai stabilizuoja kairįjį intervalą ir efektyviai siaurina dešinįjį, leisdamas rasti minimalaus taško vietą su dideliu tikslumu.

4) Niutono metodas

Skirtingai nuo ankstesnių metodų, Niutono metodas remiasi funkcijos išvestinėmis, kad greitai priartėtų prie minimumo. Pradinis taškas x_i yra koreguojamas pagal išvestinių reikšmes. Niutono metodas naudoja iteracinę formulę:

$$x_{i+1} = x_i - \frac{f'(x_i)}{f''(x_i)}$$

Šis metodas remiasi vienu pradiniu tašku x_i ir antros eilės Taylor'o plėtiniais, leidžiančiais greitai judėti link minimumo, jei pradinis taškas yra pakankamai arti minimumo. Niutono metodas paprastai veikia labai greitai ir tiksliai, bet jis reikalauja, kad funkcija būtų du kartus išvestinė.

Apskaičiavau $f'(x_i)$ ir $f''(x_i)$:

$$f(x) = \frac{x^4}{4} - 1$$

$$f'(x) = x^3$$

$$f''(x) = 3x^2$$

Parašiau Python programą (žr. Priedas 6.1, newton_method), kuri randa lokalųjį minimumo tašką naudodama Niutono metodą. Algoritmui reikėjo 25 (26 su pradiniu intervalu) iteracijų ir 25 funkcijos $f'(x)$, $f''(x)$ iškvietimų, kad būtų rastas lokalusis minimumo taškas. Lentelėje žemiau nurodžiau visų 25 (26 su pradiniu intervalu) iteracijų duomenis:

Bandymų taškai ir jų reikšmės					
Iteracija	x	$f(x)$	$f'(x)$	$f''(x)$	L
0	5	155.250000	125.000000	75.000000	10
1	3.333333	29.864198	37.037037	33.333333	1.666667
2	2.222222	5.096632	10.973937	14.814815	1.111111
3	1.481481	0.204273	3.251537	6.584362	0.740741
4	0.987654	-0.762119	0.963418	2.926383	0.493827
5	0.658436	-0.953011	0.285457	1.300615	0.329218
6	0.438957	-0.990718	0.084580	0.578051	0.219479
7	0.292638	-0.998167	0.025061	0.256912	0.146319
8	0.195092	-0.999638	0.007425	0.114183	0.097546
9	0.130061	-0.999928	0.002200	0.050748	0.065031
10	0.086708	-0.999986	0.000652	0.022555	0.043354
11	0.057805	-0.999997	0.000193	0.010024	0.028903
12	0.038537	-0.999999	0.000057	0.004455	0.019268
13	0.025691	-1.000000	0.000016	0.001980	0.012845
14	0.017127	-1.000000	0.000005	0.000880	0.0085637
15	0.011418	-1.000000	0.000001	0.000391	0.0057091
16	0.007612	-1.000000	0.0000004	0.000173	0.003806097
17	0.005075	-1.000000	0.0000001	0.000077	0.00253739
18	0.003383	-1.000000	0.00000003	0.000034	0.001691598
19	0.002255	-1.000000	0.00000001	0.000015	0.001127
20	0.001504	-1.000000	0.000000003	0.000006	0.00075182
21	0.001002	-1.000000	0.000000001	0.000003	0.00050121
22	0.000668	-1.000000	0.0000000002	0.000001	0.000334142
23	0.000446	-1.000000	0.00000000008	0.0000005	0.000222761
24	0.000297	-1.000000	0.00000000002	0.0000002	0.0001485079
25	0.000198	-1.000000	0.000000000007	0.0000001	0.0000990053201

Čia x yra bandymo taškas, o $f(x)$ jo reikšmė. $f'(x)$ tai pirmoji funkcijos $f(x)$ išvestinė, $f''(x)$ tai antroji funkcijos $f(x)$ išvestinė. L - žingsnio ilgis.

Keletas pastebėjimų, kuriuos verta paminėti:

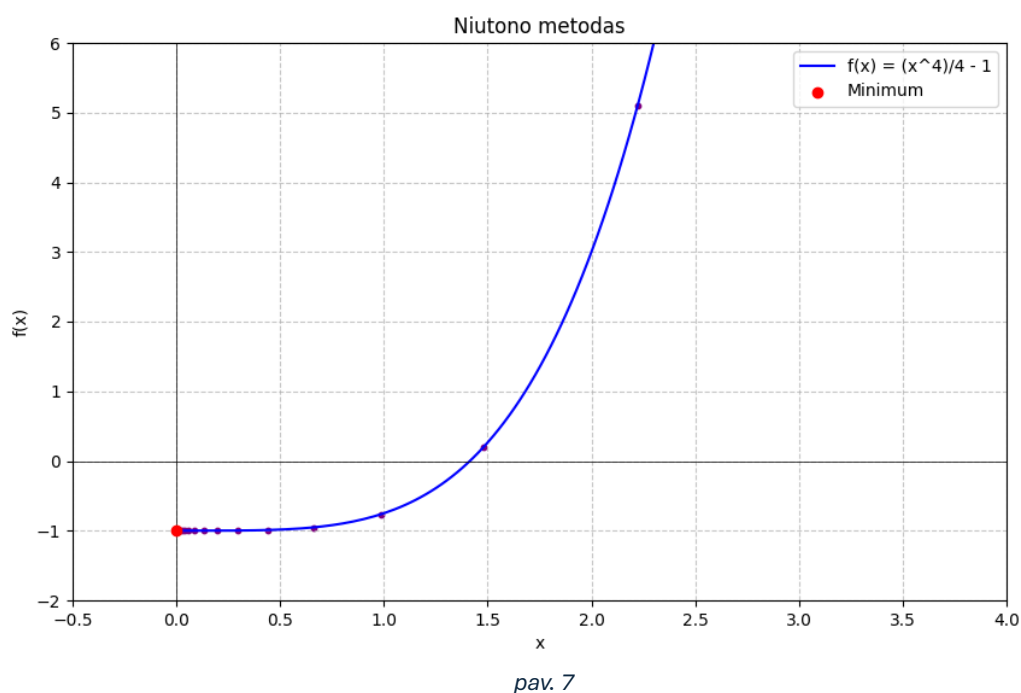
- Pradinis taškas x_1 nuo kurio pradeda veikti iteracinė formulė yra $x_1 = 5$.
- 1-oje iteracijoje $L = r - l = 10 - 0 = 10$. L reikšmė jokios įtakos 1-oje iteracijoje neturi. Ji reikalinga tik tam, kad Niutono metodas galėtų pradėti savo darbą. Nuo 2-os iteracijos L turi reikšmes: $L = x_{i-1} - x_i$, kur i - iteracijos numeris.
- Nėra 26-os iteracijos, nes 25-oje iteracijoje turime $x_{25} = 0.000198$. $L = 0,000297 - 0,000198 = 0.000099 < \varepsilon$. L mažiau už ε , todėl Niutono metodas baigė savo darbą ir gražino x_{25} , kaip lokalųjį minimumą tašką.

Niutono metodas gražino tokį $x_{\min}$ tašką:

$$x_{\min} = 0.000198$$

$$\min_{x \in [0;10]} f(x_{\min}) = \min_{x \in [0;10]} f(0.000198) \approx -1$$

Funkcijos grafikas su pavaizduotais x bandymo taškais ir minimumo reikšmė:



Šis grafikas vaizduoja Niutono metodo iteracijas ieškant funkcijos $\frac{x^4}{4} - 1$ minimumo. Mėlyna kreivė rodo pačią funkciją, o raudonas taškas žymi minimalią funkcijos reikšmę, kuri yra ties $x=0$ su $f(x) = -1$. Maži raudoni taškai vaizduoja Niutono metodo iteracijų taškus, kuriuose buvo skaičiuojami artėjant prie minimumo.

5) Išvada

Gavome tokį rezultatą(iteracijų skaičiai bė pradinio intervalo):

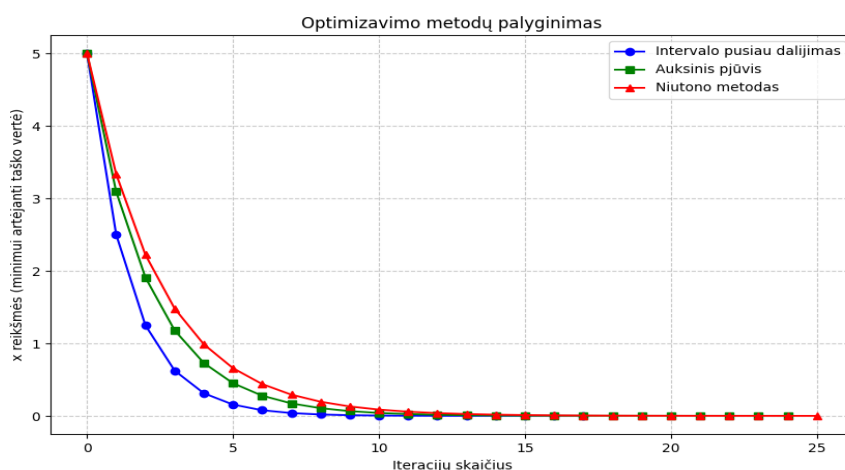
Metodas	$x_{\min}$	Iteracijų skaičius	$f(x)$ iškvietimų	$f'(x)$ iškvietimų	$f''(x)$ iškvietimų
Intervalo dalijimo pusiau	0.0000762939453	17	54	0	0
Auksinio pjūvio	0.0000482243784	24	51	0	0
Niutono	0.0001980106402	25	0	25	25

Lentelėje lyginami trys optimizavimo metodai: **intervalo dalijimo pusiau**, **auksinio pjūvio** ir **Niutono** metodai, siekiant rasti funkcijos minimumą. Pagal lentelę matyti, kad **intervalo dalijimo pusiau metodas** pasiekia sprendimą po 17 iteracijų, iškviesdamas funkciją $f(x)$ 54 kartus. Tai yra greitas ir efektyvus metodas, tačiau reikalaujantis daug $f(x)$ įvertinimų, kadangi kiekviena iteracija reikalauja įvertinti kelis intervalų taškus.

Auksinio pjūvio metodas išsprendžia užduotį po 24 iteracijų, iškviesdamas $f(x)$ 51 kartą. Nors auksinio pjūvio metodas reikalauja daugiau iteracijų nei intervalo dalijimo pusiau metodas, jis yra šiek tiek efektyvesnis pagal funkcijos $f(x)$ iškvietimų skaičių. Tai parodo, kad auksinio pjūvio metodas yra geresnis pasirinkimas, kai reikia sumažinti $f(x)$ įvertinimų kiekį, tačiau reikia daugiau iteracijų.

Niutono metodas pasiekė sprendimą per 25 iteracijas, tačiau jo išskirtinis bruožas yra tai, kad jis visiškai nenaudoja $f(x)$ įvertinimų. Vietoje to, jis pasitelkia pirmąsias ir antrąsias išvestines, iškviesdamas jas po 25 kartus. Šis metodas yra itin efektyvus, kai išvestinės galima lengvai apskaičiuoti. Nors iteracijų skaičius yra didžiausias, Niutono metodas turi privalumą, nes greitai artėja prie sprendimo, kai yra prieinamos išvestinės.

Bendrai, kiekvienas metodas turi savo stipriąsias puses: intervalo dalijimo pusiau metodas yra greitas, bet reikalauja daug $f(x)$ įvertinimų, auksinio pjūvio metodas yra balansinis, o Niutono metodas yra efektyvus, kai turime galimybę dirbti su išvestinėmis.



pav. 8

Niutono metodas, nors ir reikalauja daugiau iteracijų (25), kiekvienoje iš jų daro itin mažus žingsnius, todėl pasiekia labai aukštą tikslumo lygį. Palyginimui, auksinio pjūvio ir pusiau dalijimo metodai turi mažiau iteracijų, nes jų žingsniai yra didesni, todėl jie greičiau sumažina intervalą, tačiau jų tikslumas yra mažesnis. Ši Niutono metodo savybė leidžia jam efektyviai rasti minimumą, ypač tais atvejais, kai reikalingas didelis tikslumas.

6) Priedas (python kodas)

6.1 algorithms.py, kodas su visų algoritmų realizacijomis:

```
def f(x):
    return (x ** 4) / 4 - 1

def f_prime(x):
    return x ** 3

def f_double_prime(x):
    return 3 * x ** 2

# Interval Halving Method
def interval_halving_method(f, l, r, epsilon):
    steps = 0
    function_evals = 0
    iteration_data = []
    interval_table = []

    L = r - l

    interval_table.append({
        'Iteracija': steps,
        'l': l,
        'r': r,
        'L': L,
        'Atmetamas intervalas': None
    })

    xm = (l + r) / 2
    x1 = l + L / 4
    x2 = r - L / 4

    f_xm = f(xm)
    f_x1 = f(x1)
    f_x2 = f(x2)
    function_evals += 3

    iteration_data.append({
        'Iteration': steps,
        'x1': x1,
        'f(x1)': f_x1,
        'x2': x2,
        'f(x2)': f_x2,
        'xm': xm,
        'f(xm)': f_xm
    })

    while (r - l) > epsilon:
        steps += 1

        if f_x1 < f_xm:
```

```

        discarded_interval = 'R'
        r = xm
    elif f_x2 < f_xm:
        discarded_interval = 'L'
        l = xm
    else:
        discarded_interval = 'Both'
        l = x1
        r = x2

    L = r - l
    xm = (l + r) / 2
    x1 = l + L / 4
    x2 = r - L / 4

    f_xm = f(xm)
    f_x1 = f(x1)
    f_x2 = f(x2)
    function_evals += 3

    interval_table.append({
        'Iteracija': steps,
        'l': l,
        'r': r,
        'L': L,
        'Atmetamas intervalas': discarded_interval
    })

    iteration_data.append({
        'Iteration': steps,
        'x1': x1,
        'f(x1)': f_x1,
        'x2': x2,
        'f(x2)': f_x2,
        'xm': xm,
        'f(xm)': f_xm
    })

    return (l + r) / 2, steps, function_evals, iteration_data,
interval_table

# Golden Section Search Method
def golden_section_search(f, l, r, epsilon):
    tau = (5 ** 0.5 - 1) / 2
    steps = 0
    function_evals = 0
    L = r - l
    x1 = r - tau * L
    x2 = l + tau * L
    f_x1 = f(x1)
    f_x2 = f(x2)
    function_evals += 2

    iteration_data = []
    interval_table = []

```

```

# Pradinės reikšmės užfiksavimas
interval_table.append({
    'Iteracija': steps,
    'l': l,
    'r': r,
    'L': L,
    'Atmetamas intervalas': None
})

iteration_data.append({
    'Iteration': steps,
    'x1': x1,
    'f(x1)': f_x1,
    'x2': x2,
    'f(x2)': f_x2,
    'xm': (l + r) / 2,
    'f(xm)': f((l + r) / 2)
})

function_evals += 1

while L > epsilon:
    steps += 1

    if f_x2 < f_x1:
        discarded_interval = 'L'
        l = x1
        x1 = x2
        f_x1 = f_x2
        L = r - l
        x2 = l + tau * L
        f_x2 = f(x2)
        function_evals += 1
    else:
        discarded_interval = 'R'
        r = x2
        x2 = x1
        f_x2 = f_x1
        L = r - l
        x1 = r - tau * L
        f_x1 = f(x1)
        function_evals += 1

    interval_table.append({
        'Iteracija': steps,
        'l': l,
        'r': r,
        'L': L,
        'Atmetamas intervalas': discarded_interval
    })

    iteration_data.append({
        'Iteration': steps,
        'x1': x1,
        'f(x1)': f_x1,
        'x2': x2,
        'f(x2)': f_x2,
        'xm': (l + r) / 2,

```

```

        'f(xm)': f((l + r) / 2)
    })
    function_evals += 1

    return (l + r) / 2, steps, function_evals, iteration_data,
interval_table

# Newton's Method
def newton_method(f_prime, f_double_prime, x0, epsilon):
    steps = 0
    x = x0
    iteration_data = []

    f_x = f_prime(x)
    fpp_x = f_double_prime(x)
    L = 10
    iteration_data.append({
        'Iteration': steps,
        'x': x,
        'f(x)': f(x),
        'f\''(x)': f_x,
        'f\''\''(x)': fpp_x,
        'L': L
    })

    while True:
        if abs(f_x) <= epsilon and L <= epsilon:
            break

        x_new = x - f_x / fpp_x
        L = abs(x_new - x)
        x = x_new
        steps += 1

        f_x = f_prime(x)
        fpp_x = f_double_prime(x)

        iteration_data.append({
            'Iteration': steps,
            'x': x,
            'f(x)': f(x),
            'f\''(x)': f_x,
            'f\''\''(x)': fpp_x,
            'L': L
        })

    return x, steps, iteration_data

```

6.2 main.py, pagrindinis, vizualizacijos kodas:

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from algorithms import f, f_prime, f_double_prime,
interval_halving_method, golden_section_search, newton_method

pd.options.display.float_format = '{:.13f}'.format

interval = (0, 10)
epsilon = 1e-4
x0 = 5

# Interval Halving Method
x_min_halving, steps_halving, evals_halving, iteration_data_halving,
interval_table_halving = interval_halving_method(f, *interval, epsilon)
f_min_halving = f(x_min_halving)

# Golden Section Search
x_min_golden, steps_golden, evals_golden, iteration_data_golden,
interval_table_golden = golden_section_search(f, *interval, epsilon)
f_min_golden = f(x_min_golden)

# Newton's Method
x_min_newton, steps_newton, iteration_data_newton =
newton_method(f_prime, f_double_prime, x0, epsilon)
f_min_newton = f(x_min_newton)

results = {
    "Name": ["Intervalo pusiau dalijimas", "Auksinis pjūvis", "Niutono
metodas"],
    "Found solution": [True, True, True],
    "Solution": [x_min_halving, x_min_golden, x_min_newton],
    "Value": [f_min_halving, f_min_golden, f_min_newton],
    "Steps (N)": [steps_halving, steps_golden, steps_newton],
    "f calls": [evals_halving, evals_golden, 0],
    "df calls": [0, 0, steps_newton], # Only for Newton method
    "ddf calls": [0, 0, steps_newton] # Only for Newton method
}

x_vals = np.linspace(0, 10, 400)
y_vals = f(x_vals)

y_min, y_max = -2, 6
x_min, x_max = -0.5, 4

# Plot for Interval Halving Method
plt.figure(figsize=(10, 6))
plt.plot(x_vals, y_vals, label='f(x) = (x^4)/4 - 1', color='blue')

for data in iteration_data_halving:
    x1 = data['x1']
    x2 = data['x2']
    xm = data['xm']
```

```

plt.scatter([x1, x2], [f(x1), f(x2)], color='green', s=10, zorder=3,
label='x1 ir x2' if data['Iteration'] == 1 else "") # Green for x1 and
x2
plt.scatter(xm, f(xm), color='red', s=10, zorder=4, label='x_m' if
data['Iteration'] == 1 else "") # Red for xm

plt.scatter(x_min_halving, f_min_halving, color='red', marker='o',
zorder=5, label='Minimum')

plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
plt.ylim(y_min, y_max)
plt.xlim(x_min, x_max)
plt.xlabel('x')
plt.ylabel('f(x)')
plt.title('Intervalo pusiau dalijimas')
plt.legend(loc='upper right')
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()

plt.figure(figsize=(10, 6))
iterations = [i['Iteracija'] for i in interval_table_halving]
l_values = [i['l'] for i in interval_table_halving]
r_values = [i['r'] for i in interval_table_halving]

plt.plot(iterations, l_values, label='l (kairysis intervalas)',
color='blue', marker='o')
plt.plot(iterations, r_values, label='r (dešinysis intervalas)',
color='red', marker='o')

plt.xlabel('Iteracija')
plt.ylabel('Intervalo reikšmė')
plt.title('Intervalo kaita (Intervalo pusiau dalijimas)')
plt.legend()
plt.grid(True)
plt.show()

# Plot for Golden Section Search
plt.figure(figsize=(10, 6))
plt.plot(x_vals, y_vals, label='f(x) = (x^4)/4 - 1', color='blue')

for data in iteration_data_golden:
    x1 = data['x1']
    x2 = data['x2']
    xm = data['xm']
    plt.scatter([x1, x2], [f(x1), f(x2)], color='green', s=10, label='x1
ir x2' if data['Iteration'] == 1 else "") # Green for x1 and x2
    plt.scatter(xm, f(xm), color='red', s=10, label='x_m' if
data['Iteration'] == 1 else "") # Red for xm

plt.scatter(x_min_golden, f_min_golden, color='red', marker='o',
zorder=5, label='Minimum')

plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
plt.ylim(y_min, y_max)
plt.xlim(x_min, x_max)

```

```

plt.xlabel('x')
plt.ylabel('f(x)')
plt.title('Auksinio pjūvio metodas')
plt.legend(loc='upper right')
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()

#golden
plt.figure(figsize=(10, 6))
iterations_golden = [i['Iteracija'] for i in interval_table_golden]
l_values_golden = [i['l'] for i in interval_table_golden]
r_values_golden = [i['r'] for i in interval_table_golden]

plt.plot(iterations_golden, l_values_golden, label='l (kairysis intervalas)', color='blue', marker='o')
plt.plot(iterations_golden, r_values_golden, label='r (dešinysis intervalas)', color='red', marker='o')

plt.xlabel('Iteracija')
plt.ylabel('Intervalo reikšmė')
plt.title('Intervalo kaita (Auksinio pjūvio metodas)')
plt.legend()
plt.grid(True)
plt.show()

#dalijimas lines
plt.figure(figsize=(10, 6))
plt.plot(x_vals, y_vals, label='f(x) = (x^4)/4 - 1', color='blue')

for data in interval_table_halving:
    l = data['l']
    r = data['r']
    plt.axvline(x=l, color='green', linestyle='-', linewidth=1.5, alpha=0.7)
    plt.axvline(x=r, color='red', linestyle='-', linewidth=1.5, alpha=0.7)

plt.scatter(x_min_halving, f_min_halving, color='red', marker='o', zorder=5, label='Minimum')

plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
plt.ylim(y_min, y_max)
plt.xlim(x_min, x_max)
plt.xlabel('x')
plt.ylabel('f(x)')
plt.title('Intervalo pusiau dalijimas su intervalais')
plt.legend(loc='upper right')
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()

# golden lines
plt.figure(figsize=(10, 6))
plt.plot(x_vals, y_vals, label='f(x) = (x^4)/4 - 1', color='blue')

for data in interval_table_golden:
    l = data['l']

```

```

    r = data['r']
    plt.axvline(x=1, color='green', linestyle='-', linewidth=1.5,
alpha=0.7)
    plt.axvline(x=r, color='red', linestyle='-', linewidth=1.5,
alpha=0.7)

plt.scatter(x_min_golden, f_min_golden, color='red', marker='o',
zorder=5, label='Minimum')

plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
plt.ylim(y_min, y_max)
plt.xlim(x_min, x_max)
plt.xlabel('x')
plt.ylabel('f(x)')
plt.title('Auksinio pjūvio metodus su intervalais')
plt.legend(loc='upper right')
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()

# Plot for Newton's Method
plt.figure(figsize=(10, 6))
plt.plot(x_vals, y_vals, label='f(x) = (x^4)/4 - 1', color='blue')

x_iterations = [data['x'] for data in iteration_data_newton]
f_iterations = [f(x) for x in x_iterations]
plt.scatter(x_iterations, f_iterations, color='purple', s=10)

plt.scatter(x_min_newton, f_min_newton, color='red', marker='o',
zorder=5, label='Minimum')

plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
plt.ylim(y_min, y_max)
plt.xlim(x_min, x_max)
plt.xlabel('x')
plt.ylabel('f(x)')
plt.title('Niutono metodus')
plt.legend(loc='upper right')
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()

plt.figure(figsize=(10, 6))

steps_halving_list = [data['Iteration'] for data in
iteration_data_halving]
x_halving = [data['xm'] for data in iteration_data_halving]
plt.plot(steps_halving_list, x_halving, label='Intervalo pusiau
daliјimas', marker='o', color='blue')
steps_golden_list = [data['Iteration'] for data in iteration_data_golden]
x_golden = [data['xm'] for data in iteration_data_golden]
plt.plot(steps_golden_list, x_golden, label='Auksinis pjūvis',
marker='s', color='green')
steps_newton_list = [data['Iteration'] for data in iteration_data_newton]
x_newton = [data['x'] for data in iteration_data_newton]

```

```
plt.plot(steps_newton_list, x_newton, label='Niutono metodas',
marker='^', color='red')
plt.xlabel('Iteracijų skaičius')
plt.ylabel('x reikšmės (minimui artėjanti taško vertė)')
plt.title('Optimizavimo metodų palyginimas')
plt.legend(loc='upper right')
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()

# Summary of results
results_df = pd.DataFrame(results)
print("Summary Results:")
print(results_df.to_string(index=False))

print("\nInterval Halving Method Iterations:")
df_halving = pd.DataFrame(iteration_data_halving)
print(df_halving.to_string(index=False))

print("\nInterval Halving Method - Interval Data:")
df_interval_halving = pd.DataFrame(interval_table_halving)
print(df_interval_halving.to_string(index=False))

print("\nGolden Section Search Iterations:")
df_golden = pd.DataFrame(iteration_data_golden)
print(df_golden.to_string(index=False))

print("\nGolden Section Search - Interval Data:")
df_interval_golden = pd.DataFrame(interval_table_golden)
print(df_interval_golden.to_string(index=False))

print("\nNewton's Method Iterations:")
df_newton = pd.DataFrame(iteration_data_newton)
print(df_newton.to_string(index=False))
```